

PRISM: Structured Progress Supervision for Reward Learning from Imperfect Robot Execution

Anonymous Author(s)

Affiliation

Address

email

Abstract: Robot trajectories contain failed attempts and recovery even when they succeed, requiring reward supervision that distinguishes local setbacks from completed work. We introduce PRISM, a vision-language reward model trained on dense progress targets constructed from sparse subtask boundaries and action-quality annotations. The targets slow during suboptimal execution and reset after regression while retaining completed-subtask credit. A shared language-conditioned predictor selects the unfinished subtask, estimates local progress, and composes whole-task progress. Our annotation study finds lower time and higher protocol acceptance than direct numerical scoring; within RoboMeter, our targets improve quality ordering and failure–recovery consistency. Pretraining on 200 mostly short tasks supports zero-shot prediction on 50 held-out tasks and adaptation to two long-horizon assembly tasks. Predicted progress improves observed policy success through offline reinforcement learning and progress-weighted cloning, including when all newly collected rollouts succeed.

Keywords: Imperfect Data, Reward Learning, Robot Manipulation

Supplementary material: <https://reward-prism-corlworkshop.github.io>

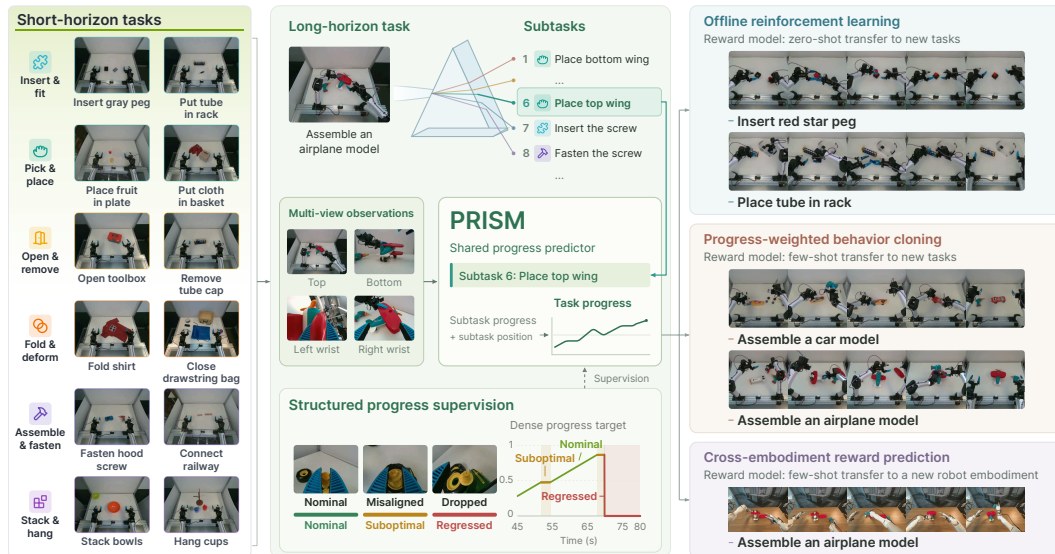


Figure 1: **Structured progress for imperfect execution.** Sparse subtask and action-quality annotations define progress targets that retain completed work through local setbacks. A shared predictor supports short-task pretraining, long-task and cross-embodiment adaptation, and reward-guided policy improvement.

1 Introduction

Robot trajectories contain repeated attempts, suboptimal actions, and failures followed by recovery, even when they succeed. Learning from them requires identifying which actions advance the task. Progress rewards provide dense signals [1, 2], but terminal outcomes cannot distinguish actions within successful trajectories, and elapsed-time targets increase through setbacks.

Consider assembling an airplane model. Misalignment should slow screw-insertion progress; dropping the screw should reset it without erasing credit for a placed wing. Recovery rebuilds insertion progress, while reaching for a drill should not advance progress until insertion is complete. This requires subtask identity and within-subtask execution quality.

PRISM combines sparse subtask boundaries with nominal, suboptimal, and regressed action-quality annotations to construct dense progress targets (Fig. 3), without asking annotators for numerical scores. We measure annotation effort and protocol acceptance, and test target utility within RoboMeter [2].

Task-specific classifiers [1] and fixed action vocabularies [3] limit reuse when subtasks change. Our shared vision-language predictor selects the unfinished subtask and estimates local progress from open-vocabulary descriptions (Fig. 2), then adds completed-subtask credit. Its language interface supports short-task pretraining and long-task adaptation without replacing components; task order defines targets for incorrect selections.

Pretraining on 200 mostly short tasks reduces progress RMSE by 20.7% against the strongest baseline on 50 held-out tasks without target-task labels or fine-tuning. Matched comparisons against base initialization show that pretraining improves adaptation on both long tasks at both budgets. Predicted progress improves policy success through offline RL and weighted cloning, including when every newly collected rollout succeeds.

Figure 1 summarizes our contributions: structured supervision for imperfect execution, a reusable compositional predictor, and evidence from annotation studies, supervision comparisons, and policy learning.

2 Related Work

Progress supervision from imperfect execution Video-derived reward supervision includes temporal ordering and transformations [4, 5, 6], signed frame distances or remaining-time targets [7, 8], and successful-transition prediction [9]. Other approaches use subtask boundaries, interval-level progress and completion labels, or simulated failures [10, 11, 12]. Our supervision combines subtask boundaries with action-quality judgments: failure resets local progress while preserving completed work, and retries rebuild progress.

Reusable and compositional reward models Visual and vision-language representations support reward signals from goals, language, and videos [13, 14, 15, 16]. Recent methods use pretrained VLMs directly [17, 18] or train across diverse tasks [19, 2]. REDS learns subtask-conditioned rewards from segmented demonstrations [20]; SARM and SARM2 estimate subtask identity and progress [1, 3]. PRM4RL composes subtask indices and local progress through simulator-state functions, while RoboMeter advances through VLM-proposed subtasks using success predictions during online RL [21, 2]. We study structured supervision for imperfect execution and reuse of a language-conditioned predictor across short tasks and longer compositions.

Reward-guided policy learning Learned visual rewards support policy improvement through RL [5, 2, 22]. Offline RL and advantage-weighted learning provide complementary tools [23, 24, 25]. We evaluate zero-shot reward models for offline RL on held-out tasks and adapted predictors for progress-weighted cloning on long tasks.

3 Method

PRISM constructs progress targets from sparse annotations and trains a shared predictor reused from short-task pretraining to long-task adaptation.

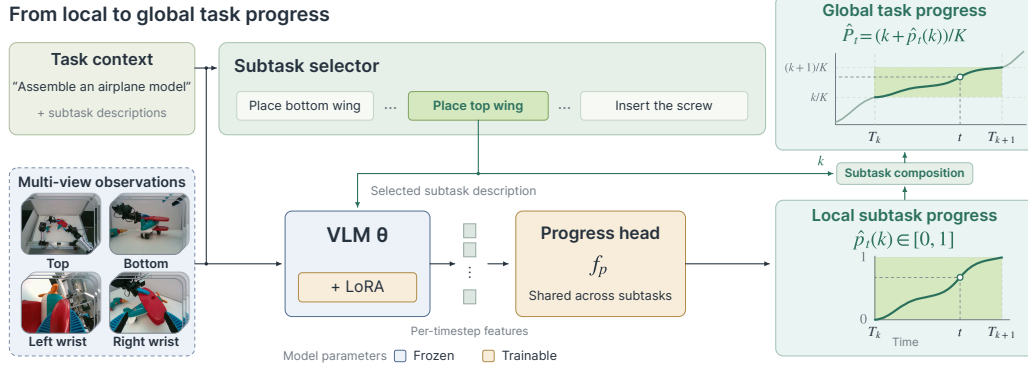


Figure 2: **From local to global progress.** A shared vision-language model selects the unfinished subtask from language descriptions and predicts its local progress. Composition adds credit for preceding completed subtasks.

3.1 Supervision for Imperfect Execution

Annotators mark subtask boundaries and *suboptimal* or *regressed* intervals; unmarked intervals are *nominal*. Suboptimal execution degrades without physical failure (e.g., a held but misaligned screw); regression loses local progress (e.g., dropping it). Each subtask receives a binary outcome (1: completed; 0: failed); completed subtasks can include regression and recovery.

For each uninterrupted attempt, nominal frames receive larger increments than suboptimal frames. In successful subtasks with retries, regression resets local progress to zero, and subsequent attempts rebuild it. The binary outcome anchors the final local target $p_t \in [0, 1]$. For the annotated zero-based subtask index k_t^* in a sequence of K subtasks, the whole-task target is $P_t^* = (k_t^* + p_t)/K$. Thus failure in subtask seven of nine retains the 6/9 credit for completed subtasks (Fig. 3). Preceding subtasks are assumed to remain completed. Without subtask anchors, accumulating delta progress [11] can fall too slowly after a failure or eventually erase completed-subtask credit.

(a) Real camera observations

Assemble an airplane model — Subtask 7/9: insert the screw



(b) Annotation methods · illustrative trajectory

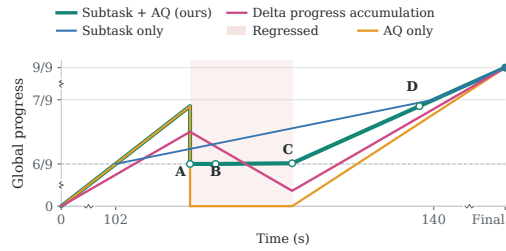


Figure 3: **Local failure and recovery.** Subtask-only targets miss the setback; AQ-only loses completed-task credit. Subtask + AQ resets insertion progress while retaining 6/9, then tracks recovery. Curves illustrate target construction.

3.2 Compositional Progress Prediction

Given multiview observations and task/ordered-subtask descriptions, the shared predictor selects the unfinished subtask’s zero-based index k_t and estimates local progress $\hat{p}_t(k_t) \in [0, 1]$ (Fig. 2). Each subtask receives an equal share of whole-task progress:

$$\hat{P}_t = \frac{k_t + \hat{p}_t(k_t)}{K}. \quad (1)$$

91 The language interface supports new subtask sequences without replacing model components, but
 92 selection errors still affect the whole-task estimate. A bidirectional temporal transformer aggregates
 93 VLM features for progress prediction. Our policy experiments score recorded trajectories offline.

94 3.3 Training the Shared Predictor

95 Selection is supervised with k_t^* . If insertion is incomplete but the model selects fastening, the latter
 96 should receive zero rather than insertion’s partial progress. Local-progress targets depend on the
 97 queried subtask, including incorrect selections:

$$y_t(k_t) = \begin{cases} 1, & k_t < k_t^*, \\ p_t, & k_t = k_t^*, \\ 0, & k_t > k_t^*. \end{cases} \quad (2)$$

98 Pretraining combines selection, annotated-subtask progress, and auxiliary whole-task progress pre-
 99 diction; the latter also uses data without reliable subtask annotations. Adaptation adds queries using
 100 the model’s own selections, with targets from Eq. 2.

101 We use Qwen3.5-4B [26] with LoRA [27], training adapters, the temporal transformer, and the
 102 progress head while freezing the base VLM. Selection and progress use distance-weighted cross-
 103 entropy. Scalar progress targets are distributed over neighboring bins; the predicted distribution’s
 104 mean yields continuous progress. Inference requires observations and task/subtask descriptions,
 105 without annotation inputs.

106 3.4 Using Progress for Policy Learning

107 For offline RL, each H -step action chunk receives the discounted sum of progress increments,

$$r_t = \sum_{j=0}^{H-1} \gamma^j [\hat{P}(s_{t+j+1}) - \hat{P}(s_{t+j})], \quad (3)$$

108 where γ is the per-step discount factor; negative changes are retained. IQL [23] learns action and
 109 state values, and their difference supplies advantages for BPPO [24] updates. For progress-weighted
 110 cloning, we map per-step changes in normalized whole-task progress, $\hat{P}(s_{t+1}) - \hat{P}(s_t)$, to high,
 111 intermediate, or low nonnegative imitation weights, with separate thresholds for human and policy
 112 actions. The weighted action loss is normalized by total weight. Each comparison holds the conversion
 113 procedure fixed across reward models. Supplementary material is available on the project website.

114 4 Experiments

115 We examine annotation effort and supervision utility, generalization across tasks and embodiments,
 116 the effects of pretraining and composition, and downstream policy learning.

117 4.1 Experimental Setup

118 Zero-shot evaluation uses 50 held-out tasks, while long-horizon adaptation uses 3–5-minute airplane
 119 model assembly ($K = 9$) and car model assembly ($K = 10$) tasks.

120 We use RMSE to measure errors in predicted progress against constructed curves that preserve
 121 completed work and capture failures and recovery. Long-task errors use subtask units (K times
 122 normalized RMSE on $[0, 1]$).

123 Policy experiments use our DreamZero World Action Model (WAM) reimplementation [28], holding
 124 initialization, data, and learning procedure fixed within each task and comparison while varying
 125 reward models.

4.2 Annotation Quality and Supervision Utility

Table 1: Annotation acceptance and time (minutes per video minute; mean $\pm$ SD).

Annotation	Accept. $\uparrow$	Time $\downarrow$
Direct points	0.76	2.6 $\pm$ 0.6
AQ only	0.90	1.4 $\pm$ 0.3
Subtask only	0.96	1.1 $\pm$ 0.2
Subtask + AQ	0.90	1.5 $\pm$ 0.4

Table 2: RoboMeter progress consistency on airplane (A) and car (C) assembly; D-U denotes Down-Up.

Supervision	Airplane		Car	
	Q-Rank $\uparrow$	D-U $\uparrow$	Q-Rank $\uparrow$	D-U $\uparrow$
Official	69.0	57.9	44.5	68.5
Original target	63.9	59.7	76.0	49.3
Our target	83.8	70.2	87.0	84.9

Annotation Interfaces We compare direct numerical scoring, subtask boundaries, action-quality (AQ) windows, and Subtask + AQ. Quality windows mark suboptimal or regressed execution; unmarked intervals are nominal. The study uses 100 trajectories and 12 annotators, each annotating every trajectory once; two additional reviewers jointly assess each annotation. Acceptance is the fraction that follows the predefined protocol without revision. Subtask + AQ raises acceptance from 0.76 to 0.90 and reduces mean annotation time from 2.6 to 1.5 minutes per video minute (Table 1).

Subtask only is faster and has higher acceptance, but boundaries alone cannot describe failures within an unfinished subtask. On clean successful trajectories without suboptimal or regressed segments, it can match Subtask + AQ given the same boundaries.

Progress Consistency under Different Supervision We evaluate RoboMeter using human-labeled quality ordering (Q-Rank) and score decreases followed by increases around regressed events (Down-Up). Constructed-target finetuning achieves the highest scores on both tasks (Table 2).

4.3 Reward Prediction and Generalization

Zero-Shot Prediction and Pretraining

On the 50 held-out tasks without target-task reward labels or fine-tuning, PRISM achieves RMSE 0.180 versus 0.227 for the strongest baseline, RoboMeter, a 20.7% reduction (Fig. 4(a)). Panel (a) uses shared 200-task training, with released RoboMeter as an external reference. RoboMeter fine-tuning starts from its released checkpoint. Increasing pretraining from 15 to 200 tasks reduces RMSE on the same held-out tasks by 35.3% (Fig. 4(b)). The 15-, 30-, and 200-task sets are nested, with compute and other settings fixed. Budgets are displayed as equally spaced conditions; the panels use different y-axis scales.

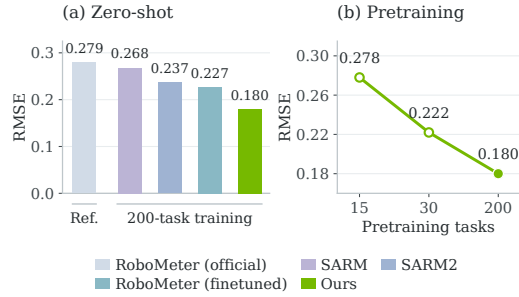


Figure 4: **Held-out progress RMSE.** (a) Zero-shot evaluation. (b) Pretraining scale.

Long-Horizon Adaptation We adapt each model to the plane-model and car-model assembly using 15 (D15) or 100 (D100) training trajectories per task, sharing annotations and constructed global progress targets. PRISM achieves the lowest RMSE in all four task-budget settings and, with D15, outperforms every baseline using D100 (Table 3). Averaged in subtask units across tasks, RMSE is 37.7% below SARM2, the strongest baseline, at both budgets. Section 4.3 analyzes pretrained initialization.

Cross-Embodiment Adaptation We transfer reward models fine-tuned for airplane model assembly on the YAM robot to the same task on the Tianji robot, using 10 Tianji trajectories, and evaluate them on the same 30 Tianji test trajectories. PRISM reaches RMSE 0.452 versus 0.556 for the strongest baseline, RoboMeter, an 18.7% reduction (Table 4).

Table 3: Long-task RMSE in subtask units. D15/D100 use 15/100 labeled trajectories.

Model	Airplane ($K = 9$)		Car ($K = 10$)	
	D15	D100	D15	D100
RoboMeter	0.493	0.487	0.658	0.549
SARM	0.747	0.485	1.124	0.539
SARM2	0.498	0.461	0.629	0.472
Ours: no-pretrain	0.382	0.311	0.502	0.445
Ours	0.308	0.262	0.394	0.319

Table 4: YAM to Tianji RMSE (0–9); 10 labeled trajectories.

Model	RMSE
SARM	0.919
SARM2	1.167
RoboMeter	0.556
Ours	0.452

Reusing Pretrained Knowledge PRISM reuses its complete pretrained predictor through the same language interface, whereas SARM replaces its task-specific classifier and SARM2 requires manual vocabulary mapping for full reuse. All models receive subtask descriptions and target-task training; PRISM needs no additional task-interface changes. Full SARM reuse is unsupported, and “Better” counts improvements among four task–budget settings. For PRISM, pretrained and base Qwen3.5-4B initializations share the architecture, adaptation data and procedure, and evaluation.

Pretraining reduces PRISM’s RMSE by 15.8–28.3% in all four settings, versus improvements in two or three settings for the baselines (Table 5).

Table 5: Predictor reuse: relative RMSE change from each method’s baseline without reward-model pretraining (%; negative is better). “Better” counts improvements across four settings.

Reuse	Model	Airplane		Car		Better
		D15	D100	D15	D100	
Full predictor	Ours	−19.4	−15.8	−21.5	−28.3	4/4
	SARM	—	—	—	—	N/A
	SARM2	−9.7	+4.4	−18.2	+2.2	2/4
Adapt classifier	SARM	+24.4	−28.7	−10.0	−1.8	3/4
	SARM2	−2.9	+22.2	−9.1	+15.6	2/4

Public-Benchmark Trajectory Ranking Following RBM-EVAL-OOD [2], we fine-tune on RBM-1M with progress-only supervision and report Kendall scores for trajectory ordering (failure < suboptimal < successful).

Without reward-model pretraining, switching the backbone from Qwen3-VL to Qwen3.5-4B improves the macro ranking score only slightly, from 0.519 to 0.529 (Table 6). Keeping Qwen3.5-4B fixed, pretraining with dense progress supervision on data from a different distribution brings a much larger gain, from 0.529 to 0.688, after the same progress-only fine-tuning on RBM-1M. Both RoboMeter rows, including per-dataset values, are quoted from its paper.

Table 6: RBM-EVAL-OOD ranking. Macro averages six domains; RoboMeter results quote arXiv v1 [2].

Method / initialization	USC-F	USC-K	USC-T	USC-X	MIT-F	UTD-S	Macro↑
<i>mixed-objective supervision</i>							
RoboMeter: official [2]	0.646	0.471	0.653	0.694	0.601	0.867	0.655
<i>progress-only supervision</i>							
RoboMeter: progress-only [2]	0.083	0.231	0.333	0.389	0.183	0.533	0.292
Ours: no-pretrain (Qwen3-VL)	0.333	0.456	0.389	0.500	0.504	0.933	0.519
Ours: no-pretrain (Qwen3.5-4B)	0.333	0.456	0.389	0.500	0.560	0.933	0.529
Ours	0.583	0.560	0.917	0.639	0.496	0.933	0.688

180 4.4 Subtask Composition and Progress Prediction

181 Subtask Composition

182 On airplane model assembly, direct whole-task and Table 7: Composition on airplane assembly.
183 composed predictors start from base Qwen3.5-4B RMSE in subtask units; D15/D100 labeled tra-
184 without reward-model pretraining and use matched jectories.
185 adaptation budgets. Automatic subtask selection
186 and local-progress composition reduce RMSE by
187 17.3% with D15 and 13.4% with D100, including
188 selection errors (Table 7).

Configuration	D15↓	D100↓
Direct whole-task	0.462	0.359
Subtask composition	0.382	0.311

189 In zero-shot car model assembly, composition reduces accumulated RMSE from 1.578 to 0.524 for
190 GPT-6 Astra [29] and by 32.4% for base Qwen3.5-4B (Table 8). In our setup, GPT-6 compositional
191 scoring costs approximately US\$38.08 per trajectory at 1 fps for trajectories averaging 340 seconds;
192 PRISM runs locally on one GPU with more than 10 GB VRAM, avoiding recurring API fees.

193 **Selection and Local Progress** Using its own subtask predictions without car model adaptation,
194 pretrained PRISM reaches 69.46% subtask selection accuracy and RMSE 1.3803, versus 25.37% and
195 3.0676 for base Qwen3.5-4B with composition. Holding GPT-6’s subtask predictions fixed, replacing
196 its local-progress estimates with those from pretrained PRISM further reduces accumulated RMSE
197 from 0.524 to 0.494, without car model adaptation (Table 8). This highlights the value of the learned
198 local-progress predictor under the same subtask selector.

Table 8: Car-model selection accuracy and progress RMSE. Base Qwen is Qwen3.5-4B; zero-shot Ours uses reward-model pretraining.

Model / procedure	Acc.↑	RMSE↓
<i>Zero-shot</i>		
GPT-6: direct	—	1.578
GPT-6: composition	86.56%	0.524
Base Qwen: direct	—	4.541
Base Qwen: composition	25.37%	3.068
Ours: self-selected	69.46%	1.380
Ours: GPT-selected	86.56%	0.494
RoboMeter: GPT-selected	86.56%	0.624
<i>Task-related adaptation (15 trajs)</i>		
Ours: no-pretrain + adapt.	93.33%	0.502
Ours: pretrained + adapt.	94.70%	0.394

4.5 Downstream Policy Improvement

Offline RL with Zero-Shot Rewards

Each reward source scores the same BC-policy rollouts on two held-out tasks: 50 for star insertion (one subtask) and 45 for tube transfer from a box to a rack (three subtasks). For offline RL, the same reward-conversion rule supplies IQL critic learning [23] and BPPO actor updates [24]. Zero-shot means reward models receive no target-task reward annotations; policies train on scored target-task rollouts.

PRISM yields the highest observed success, 65.0% on star insertion and 66.7% on tube transfer, versus 50.0% and 41.7% for the strongest competing reward sources (Fig. 5). Across the four reward sources shared with Fig. 4(a), lower aggregate zero-shot RMSE on the 50 held-out tasks corresponds to higher or tied success on both downstream tasks.

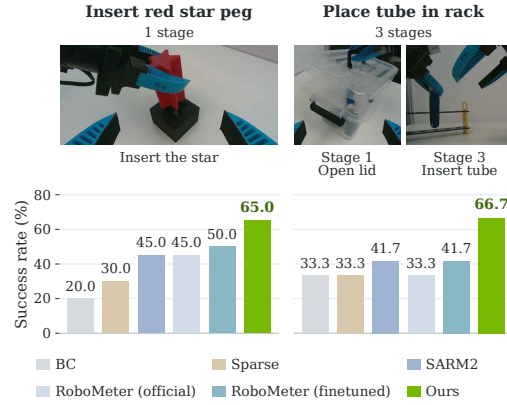


Figure 5: **Offline policy learning.** Zero-shot reward models; 20 evaluation trials for star insertion and 12 for tube transfer; shared task-specific training rollouts.

Progress-Weighted Cloning

On airplane and car assembly, adapted reward models weight recorded actions. We compare PRISM, unweighted BC, and SARM2 (the strongest baseline by average long-task RMSE), keeping the progress-difference weighting procedure fixed (Sec. 3.4) following SARM [1].

Iter1 starts from a task-specific BC policy trained on 20 demonstrations (25% success). Each condition trains on these demonstrations plus 12 shared DAGger trajectories. PRISM reaches 58% success on both tasks, versus 42% for unweighted BC and 33%/42% for SARM2 on airplane/car assembly.

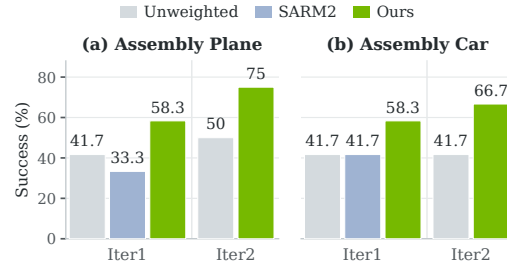


Figure 6: **Progress-weighted learning.** Each success rate uses 12 trials. Conditions share a task-specific parent policy within each iteration.

Iter2 uses each task’s highest-success Iter1 checkpoint (58%) as the shared parent. Both conditions train on the original 20 demonstrations and 12 newly collected successful rollouts from this parent, excluding Iter1 DAGger data; PRISM uses the same weighting procedure. Weighted cloning achieves 75% versus 50% on airplane assembly and 67% versus 42% on car assembly (Fig. 6). Since all 12 new rollouts succeed, terminal outcomes cannot distinguish useful from unhelpful chunks within them; the results support dense progress changes for local credit assignment.

5 Conclusion

PRISM combines structured progress supervision for setbacks and recovery with a shared predictor. Its targets improve RoboMeter’s progress consistency; short-task pretraining improves long-task adaptation under matched comparisons and supports zero-shot prediction and public OOD ranking after adaptation. Predicted progress improves policies through offline RL and progress-weighted cloning, including when all newly collected rollouts succeed.

References

- [1] Q. Chen, J. Yu, M. Schwager, P. Abbeel, F. Shentu, and P. Wu. SARM: Stage-aware reward modeling for long horizon robot manipulation, 2025.
- [2] A. Liang, Y. Korkmaz, J. Zhang, M. Hwang, A. Anwar, S. Kaushik, A. Shah, A. S. Huang, L. Zettlemoyer, D. Fox, Y. Xiang, A. Li, A. Bobu, A. Gupta, S. Tu, E. Biyik, and J. Zhang. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons, 2026.
- [3] Q. Chen, H. Zheng, J. Yu, S. Huang, J. Sun, K. Goldberg, C. Wen, P. Abbeel, Y. Shentu, P. Wu, and M. Schwager. SARM2: Multi-task stage aware reward modeling for self improving robotic manipulation, 2026.
- [4] M. Alakuijala, R. McLean, I. Woungang, N. Farsad, S. Kaski, P. Marttinen, and K. Yuan. Video-Language Critic: Transferable Reward Functions for Language-Conditioned Robotics. *Transactions on Machine Learning Research*, 2025.
- [5] J. Zhang, Y. Luo, A. Anwar, S. A. Sontakke, J. J. Lim, J. Thomason, E. Biyik, and J. Zhang. ReWiND: Language-guided rewards teach robot policies without new demonstrations, 2025.
- [6] J. Yu, A. Goldberg, K. Kondap, K. El-Refai, E. Ransing, Q. Chen, M. Schwager, F. Shentu, P. Wu, and K. Goldberg. WARP-RM: A warp-augmented relative progress reward model for data curation, 2026.
- [7] Y. Liu, C. Wen, Y. Hu, D. Jayaraman, and Y. Gao. TimeRewarder: Learning dense reward from passive videos via frame-wise temporal distance, 2026. ICML 2026 spotlight; first preprint 2025.
- [8] D. Huang, H. Zhang, B. Hou, S. Huang, Z. Su, H. Guo, T. Lu, Z. Xu, J. Tang, J. Yang, D. Wang, P. Peng, M. Chen, D. Zhao, and X. Li. RynnValue: Scaling robotic value foundation models with temporal distance, 2026.
- [9] H. Zhang, Z. Zeng, B. Wang, L. Li, L. Lin, and L. Cheng. Dream2Reward: Transition-Alignment Reward Models from Positive Demonstrations for Robotic Manipulation, 2026.
- [10] H. Tan, S. Chen, Y. Xu, Z. Wang, Y. Ji, C. Chi, Y. Lyu, Z. Zhao, X. Chen, P. Co, S. Xie, G. Yao, P. Wang, Z. Wang, and S. Zhang. Robo-Dopamine: General process reward modeling for high-precision robotic manipulation, 2025.
- [11] Y. Mao, Z. Yu, W. Mao, Y. Li, Q. Hu, Z. Lan, M. Zhu, and H. Chen. ARM: Advantage reward modeling for long-horizon manipulation, 2026.
- [12] Y. Fang, W. Dong, J. Liu, Y. Yang, M. Huo, Y. Mu, H. Yao, L. E. Li, D. Szafrir, and M. Ding. DenseReward: Dense reward learning via failure synthesis for robotic manipulation, 2026.
- [13] Y. J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, and A. Zhang. VIP: Towards Universal Visual Reward and Representation via Value-Implicit Pre-Training. In *International Conference on Learning Representations*, 2023.
- [14] Y. J. Ma, W. Liang, V. Som, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman. Liv: Language-image representations and rewards for robotic control, 2023.
- [15] J. Rocamonde, V. Montesinos, E. Nava, E. Perez, and D. Lindner. Vision-Language Models are Zero-Shot Reward Models for Reinforcement Learning. In *International Conference on Learning Representations*, 2024.
- [16] S. Sontakke, J. Zhang, S. Arnold, K. Pertsch, E. Biyik, D. Sadigh, C. Finn, and L. Itti. RoboCLIP: One Demonstration is Enough to Learn Robot Policies. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

- [17] Y. J. Ma, J. Hejna, A. Wahid, C. Fu, D. Shah, J. Liang, Z. Xu, S. Kirmani, P. Xu, D. Driess, T. Xiao, J. Tompson, O. Bastani, D. Jayaraman, W. Yu, T. Zhang, D. Sadigh, and F. Xia. Vision language models are in-context value learners, 2024.
- [18] S. Chen, C. Harrison, Y.-C. Lee, A. J. Yang, Z. Ren, L. J. Ratliff, J. Duan, D. Fox, and R. Krishna. TOPReward: Token probabilities as hidden zero-shot rewards for robotics, 2026.
- [19] T. Lee, A. Wagenmaker, K. Pertsch, P. Liang, S. Levine, and C. Finn. RoboReward: General-purpose vision-language reward models for robotics, 2026.
- [20] C. Kim, M. Heo, D. Lee, H. Lee, J. Shin, J. J. Lim, and K. Lee. Subtask-aware visual reward learning from segmented demonstrations. In *International Conference on Learning Representations*, 2025.
- [21] X. Zhang, N. Gao, X. Jiang, Y. Chen, Y. Pan, M. Zhang, and Y. Deng. Progress reward model for reinforcement learning via large language models. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 100745–100772, 2025. doi:10.52202/085713-3370.
- [22] S. Zhai, Q. Zhang, T. Zhang, F. Huang, H. Zhang, M. Zhou, S. Zhang, L. Liu, S. Lin, and J. Pang. A Vision-Language-Action-Critic Model for Robotic Real-World Reinforcement Learning, 2025.
- [23] I. Kostrikov, A. Nair, and S. Levine. Offline Reinforcement Learning with Implicit Q-Learning. In *International Conference on Learning Representations*, 2022.
- [24] Z. Zhuang, K. Lei, J. Liu, D. Wang, and Y. Guo. Behavior Proximal Policy Optimization. *arXiv preprint arXiv:2302.11312*, 2023.
- [25] X. B. Peng, A. Kumar, G. Zhang, and S. Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning, 2019.
- [26] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026.
- [27] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [28] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, A. Malik, K. Lee, W. Liang, N. Ranawaka, J. Gu, Y. Xu, G. Wang, F. Hu, A. Narayan, J. Bjorck, J. Wang, G. Kim, D. Niu, R. Zheng, Y. Xie, J. Wu, Q. Wang, R. Julian, D. Xu, Y. Du, Y. Chebotar, S. Reed, J. Kautz, Y. Zhu, L. J. Fan, and J. Jang. World Action Models are Zero-shot Policies, 2026.
- [29] OpenAI. GPT-6 Astra. OpenAI API documentation, 2026. Accessed: 2026-09-16.